

Structured Text

(ST) Programming

Variables | IF/ELSE | WHILE | FOR | CASE | Functions

What You Will Learn

- ST syntax: PROGRAM structure, variable declarations, assignment operator :=
- Control flow: IF/ELSIF/ELSE, WHILE, FOR, REPEAT, CASE — when to use each
- Calling standard function blocks (TON, CTU) and writing your own FUNCTION_BLOCKS
- Real ST programmes: analogue scaling, recipe system, alarm manager

```
IF Temperature > 85.0 THEN
    Alarm_High := TRUE;
ELSIF Temperature < 10.0 THEN
    Alarm_Low := TRUE;
END_IF;
```

Awet G. Nway

Founder, AYE Tech Hub | PLC & Industrial Automation Engineer

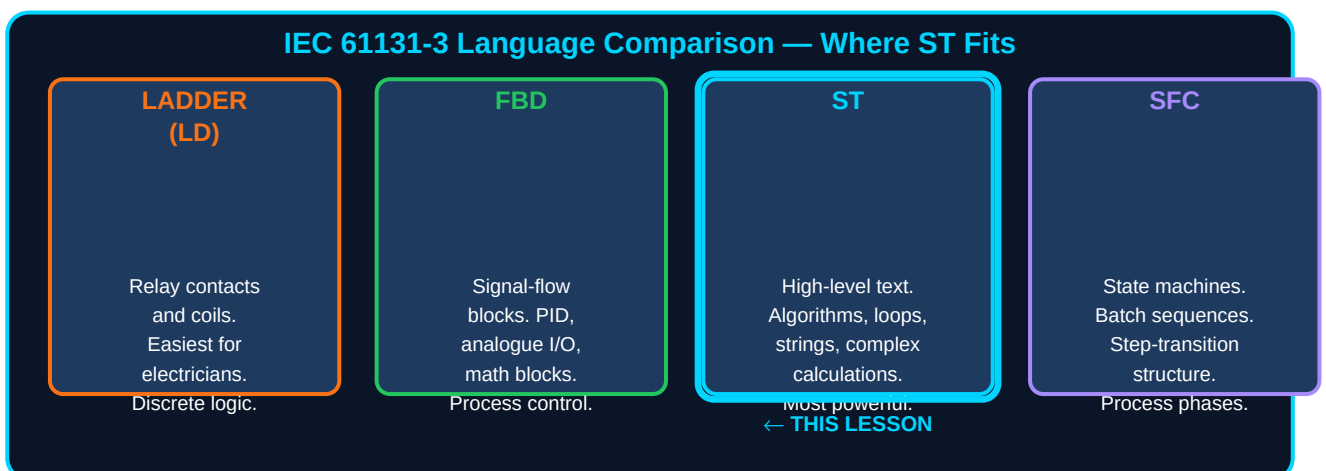
PLC-009: Structured Text (ST) Programming

Structured Text is the most powerful of the five IEC 61131-3 PLC programming languages. Unlike Ladder Diagram (which looks like a relay wiring diagram) or FBD (which looks like a signal-flow block diagram), ST looks like a conventional high-level programming language — similar to Pascal or C. This makes it the natural choice for complex algorithms, mathematical calculations, string processing, recipe management, and any logic that would require dozens of Ladder rungs to express clearly.

Learning Objectives

- Write ST programs with correct PROGRAM/END_PROGRAM structure and VAR declarations
- Use the five ST data types: BOOL, INT, REAL, TIME, and STRING with correct assignment syntax
- Implement IF/ELSIF/ELSE, WHILE, FOR, REPEAT, and CASE control flow structures
- Call standard function blocks (TON, CTU) and write user-defined FUNCTION_BLOCKS in ST
- Apply ST to real engineering tasks: analogue scaling, recipe systems, and alarm management

1. Where ST Fits in IEC 61131-3



2. ST Program Structure

Every ST programme in Siemens TIA Portal (or any IEC 61131-3 platform) follows the same structure: variable declarations at the top, code below. In TIA Portal, ST is written inside Organisations Blocks (OBs), Functions (FCs), or Function Blocks (FBs).

```
(* PLC-009 example: ST program structure *)  
PROGRAM Motor_Control_ST  
VAR  
PB_Start : BOOL; (* Start pushbutton, NO *)  
PB_Stop  : BOOL; (* Stop pushbutton, NC *)  
Motor_Run : BOOL; (* Motor output coil *)
```

```
Run_Timer : TON; (* On-delay timer instance *)
Run_Hours : DINT := 0; (* Accumulated runtime hours *)
END_VAR

(* Control logic *)
IF PB_Start AND NOT PB_Stop THEN
  Motor_Run := TRUE;
ELSIF NOT PB_Stop = FALSE THEN
  Motor_Run := FALSE;
END_IF;
```

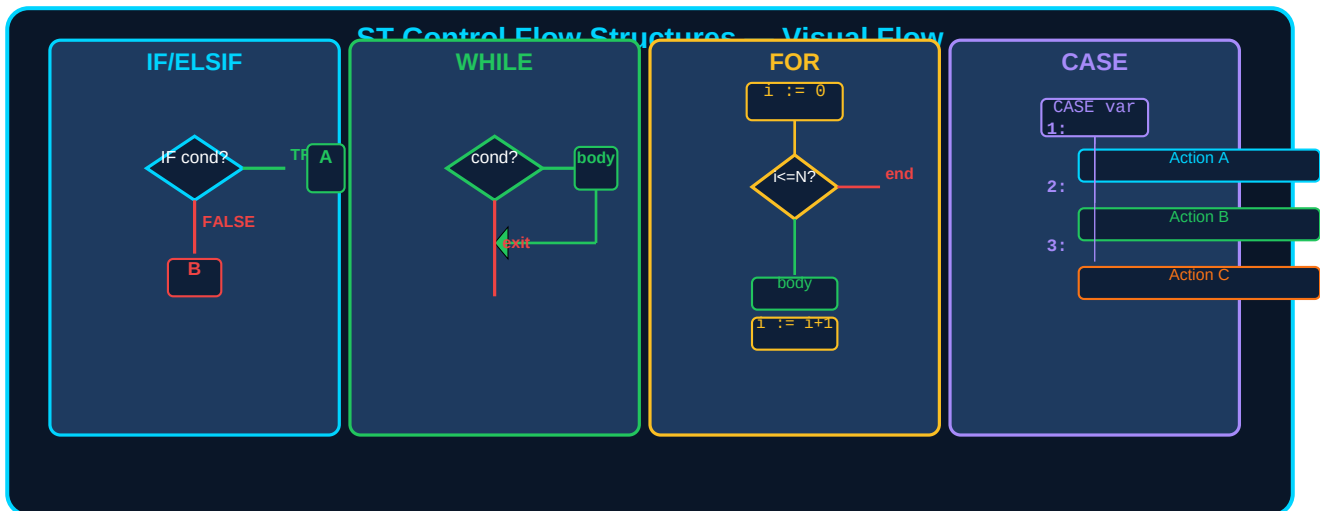
3. Variables and Data Types

Data Type	Size	Range / Values	Declaration Example	Common Use
BOOL	1 bit	TRUE or FALSE	Run_OK : BOOL := FALSE;	Digital I/O, flags, status bits
INT	16 bit	−32,768 to +32,767	Counter_Val : INT := 0;	Loop counters, small integers
DINT	32 bit	−2.1B to +2.1B	Runtime_s : DINT := 0;	Large counters, accumulated values
REAL	32 bit	±1.18E−38 to ±3.40E+38	Temperature : REAL := 0.0;	Analogue values, scaled readings
TIME	32 bit	T#0ms to T#49d17h...	Delay_Time : TIME := T#5s;	Timer preset values
STRING	Variable	Up to 254 chars	Alarm_Msg : STRING;	Alarm messages, HMI text
ARRAY	Variable	Fixed-size indexed array	Recipe : ARRAY[0..9] OF REAL;	Recipe tables, data logging

ST Operators — Complete Reference

Category	Operators	Example
Assignment	:= (assign value to variable)	Speed_SP := 1500.0;
Arithmetic	+ − * / MOD (modulo)	Result := (A + B) * C / D;
Comparison	= < > <= >=	IF Temperature >= 80.0 THEN
Logical (BOOL)	AND OR NOT XOR	IF PB_Start AND NOT E_Stop THEN
Bitwise	AND OR XOR NOT (on integers)	Status_Word := Input_W AND 16#00FF;
Type conversion	INT_TO_REAL REAL_TO_INT BOOL_TO_INT	EU_Value := INT_TO_REAL(Raw_Value);

4. Control Flow Structures



5. IF / ELSIF / ELSE / END_IF

```
(* Temperature alarm with multiple thresholds *)  
IF Temperature > 95.0 THEN  
  Alarm_Critical := TRUE;  
  Alarm_High := TRUE;  
  Motor_Run := FALSE; (* Emergency stop *)  
ELSIF Temperature > 80.0 THEN  
  Alarm_High := TRUE;  
  Alarm_Critical := FALSE;  
ELSIF Temperature < 10.0 THEN  
  Alarm_Low := TRUE;  
  Alarm_High := FALSE;  
  Alarm_Critical := FALSE;  
ELSE  
  Alarm_High := FALSE;  
  Alarm_Low := FALSE;  
  Alarm_Critical := FALSE;  
END_IF;
```

6. FOR Loop — Array Processing and Counting

```
(* Find maximum value in a 10-element array *)  
VAR  
Data_Array : ARRAY[0..9] OF REAL;  
Max_Value : REAL := -99999.0;  
i : INT;  
END_VAR  
FOR i := 0 TO 9 DO  
  IF Data_Array[i] > Max_Value THEN  
    Max_Value := Data_Array[i];  
  END_IF;  
END_FOR;  
  
(* Recipe loading: copy 5 parameters from recipe array *)  
FOR i := 0 TO 4 DO  
  Active_Recipe[i] := Recipe_Bank[Recipe_Num, i];  
END_FOR;
```

7. CASE — State Machines and Mode Control

```
(* Production machine state machine *)  
CASE Machine_State OF  
  0: (* IDLE *)  
    Run_Output := FALSE;
```

```
IF Start_PB AND Parts_Ready THEN
Machine_State := 1;
END_IF;

1: (* RUNNING *)
Run_Output := TRUE;
Part_Counter := Part_Counter + 1;
IF E_Stop OR NOT Parts_Ready THEN
Machine_State := 2;
ELSIF Part_Counter >= Batch_Size THEN
Machine_State := 3;
END_IF;

2: (* FAULT *)
Run_Output := FALSE;
Fault_Light := TRUE;
IF Reset_PB AND NOT E_Stop THEN
Machine_State := 0;
Fault_Light := FALSE;
END_IF;

3: (* BATCH COMPLETE *)
Run_Output := FALSE;
Batch_Done := TRUE;
Part_Counter := 0;
Machine_State := 0;
END_CASE;
```

8. Calling Standard Function Blocks in ST

In ST, function blocks are called using their instance name followed by parentheses with named parameters. The FB instance must be declared in VAR.

```
(* Declare instances in VAR section *)
VAR
Pump_Timer : TON; (* Timer instance *)
Part_Count : CTU; (* Counter instance *)
END_VAR

(* Call TON timer – call every scan *)
Pump_Timer(IN := Pump_Running, PT := T#30m);

(* Check timer output *)
IF Pump_Timer.Q THEN
Pump_Service_Due := TRUE;
END_IF;

(* Call CTU counter *)
Part_Count(CU := Part_Sensor, R := Reset_Button, PV := 500);

IF Part_Count.Q THEN
Batch_Complete := TRUE;
END_IF;

Batch_Count_Display := Part_Count.CV; (* Current value to HMI *)
```

9. Analogue Scaling in ST — Real-World Application

Converting a raw 4–20 mA analogue input to engineering units is one of the most common ST tasks. The formula: $EU = (Raw / 27648) \times Span + EU_Min$.

```
(* Complete analogue scaling function in ST *)
(* Siemens S7: 4mA = 0, 20mA = 27648 *)

VAR
AI_Raw : INT; (* Raw analogue input value *)
Raw_Real : REAL; (* Converted to REAL for calculation *)
Temperature : REAL; (* Scaled engineering value in °C *)
Temp_EU_Min : REAL := 0.0; (* 0°C at 4 mA *)
Temp_EU_Max : REAL := 150.0; (* 150°C at 20 mA *)
END_VAR

(* Step 1: Convert INT to REAL *)
Raw_Real := INT_TO_REAL(AI_Raw);

(* Step 2: Apply scaling formula *)
Temperature := (Raw_Real / 27648.0)
* (Temp_EU_Max - Temp_EU_Min)
+ Temp_EU_Min;

(* Step 3: Limit output to valid range (clamp) *)
```

```
IF Temperature > Temp_EU_Max THEN
Temperature := Temp_EU_Max;
ELSIF Temperature < Temp_EU_Min THEN
Temperature := Temp_EU_Min;
END_IF;
```

10. Practice Exercises

#	Exercise	Instructions
1	Variable declaration and assignment	Declare variables for: a motor speed setpoint (REAL, default 1500.0), a part counter (DINT, default 0), a run status flag (BOOL, default FALSE), and a 10-element REAL array for sensor readings. Write ST code to: set speed to 750.0, increment counter, set flag TRUE if counter > 100.
2	IF temperature alarm	Write an IF/ELSIF/ELSE structure for a boiler temperature alarm: Above 95°C → Critical alarm + shutdown. 80–95°C → High alarm, continue. 10–80°C → Normal, clear alarms. Below 10°C → Low alarm. Use REAL comparisons with correct ST syntax.
3	FOR loop — find average	Declare an ARRAY[0..7] OF REAL named Readings. Write a FOR loop that sums all 8 values into a REAL variable Sum, then calculates Average := Sum / 8.0. Add error handling: if any value is negative, skip it and reduce the divisor accordingly.
4	CASE state machine	Implement a simple traffic light controller as a CASE state machine: State 0: Red (10 seconds). State 1: Red+Amber (2 seconds). State 2: Green (8 seconds). State 3: Amber (3 seconds). Use a TON timer and cycle through states automatically.
5	Analogue scaling + alarm	Scale a pressure transmitter: raw 0–27648, engineering units 0–16 bar. Write ST code to: scale the raw value, apply Hi alarm at 14 bar, Lo alarm at 2 bar, HiHi shutdown at 15 bar. Include hysteresis: alarm clears when value returns to 80% of alarm setpoint.

Lesson Summary

Key Concept	Summary Rule
ST structure	VAR declarations at top (PROGRAM or FUNCTION_BLOCK block). Code below. END_PROGRAM or END_FUNCTION_BLOCK at bottom.
Assignment operator	:= (colon equals) assigns values. = is comparison. Never confuse the two — := always for assignment in ST.
IF statement	IF ... THEN ... ELSIF ... THEN ... ELSE ... END_IF. Each condition ends with THEN. Block always ends with END_IF.
FOR loop	FOR i := start TO end BY step DO ... END_FOR. BY 1 is default. Use for counting, array iteration, recipe loading.
WHILE loop	WHILE condition DO ... END_WHILE. Check condition before each iteration. Risk of infinite loop if condition never becomes FALSE.
CASE statement	CASE var OF 1: ... 2: ... ELSE: ... END_CASE. Cleanest way to implement state machines and mode selection.
Calling FBs	Declare instance in VAR. Call every scan: Instance_Name(IN := value, PT := T#5s). Access outputs: Instance.Q, Instance.ET.

Analogue scaling	$EU = (INT_TO_REAL(Raw) / 27648.0) \times \text{Span} + EU_Min.$ Always convert INT to REAL first before division.
When to use ST vs LD	ST: complex maths, algorithms, loops, arrays, strings, state machines. LD: simple discrete I/O logic, relay replacement.

What Is Next — PLC-010

PLC-010: Sequential Function Chart (SFC) — The graphical language for step-by-step sequential control. Steps, transitions, and branches. Actions attached to steps. Simultaneous and alternative sequences. Batch process control, machine cycles, and wash-down sequences. Implementing SFC in Siemens TIA Portal and Allen-Bradley Studio 5000.

Resource	Details
PLC PDF Series	PLC-001 through PLC-030 — ayetechub.com/pdfs.html
Free PLC Simulator	Browser Ladder Logic tool — ayetechub.com/downloads
Community	Telegram: t.me/ayetechub